Design a URL Shortener (TinyURL)

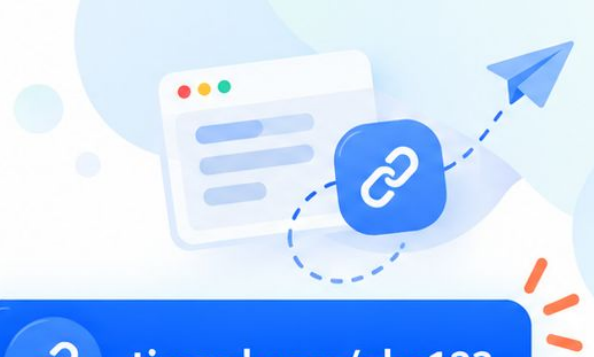
Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Introduction to TinyURL

Turn long web addresses into short, shareable links.



https://www.example.com/very/long/path/to/resource → tinyurl.com/abc123

Why it's useful

Compact links
Perfect for social platforms like Twitter and SMS.

Improved UX
Easy to share, clean, and readable.

Click tracking
Analytics for link performance.

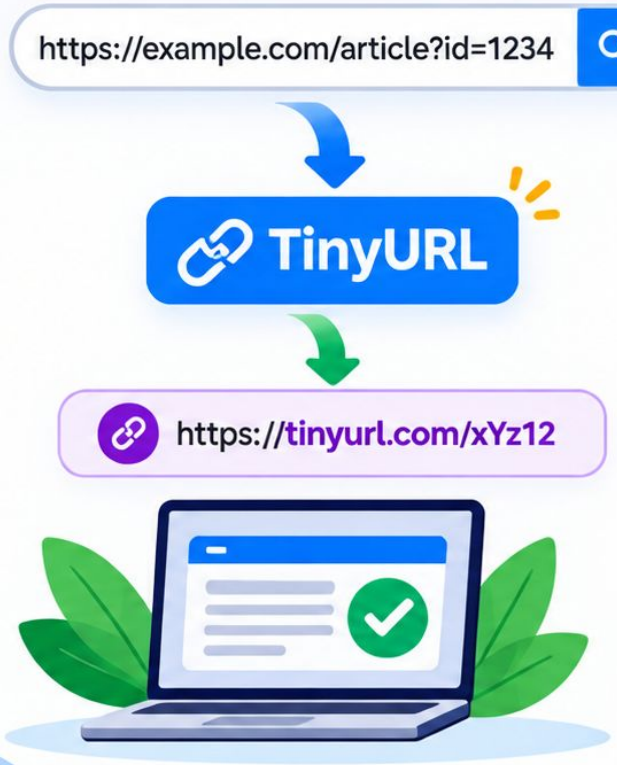
Branding
Custom domains for enterprises.

Email | Chat | Print | Works across email, chat & print.

How it works

1. **Submit URL** — User submits a long URL.
2. **Generate key** — System generates a unique short key.
3. **Store mapping** — Save short key and original URL in database.
4. **Redirect** — Short URL sends visitor to original URL.

Functional Requirements for TinyURL



1. **Shorten a URL**
   Accept a valid long URL and return a shortened URL.
   Example: Input: https://example.com/article?id=1234 → Output: https://tinyurl.com/xYz12

2. **Redirect to Original URL**
   When accessing the short URL, redirect to the original long URL.

3. **Prevent Duplicate Short URLs**
   If the same long URL is submitted, return the same short URL or handle according to configuration (unless custom alias is used).

4. **User Authentication**
   Allow users to register/login to manage URLs, view analytics, and set expiration.

Non-Functional Requirements for TinyURL





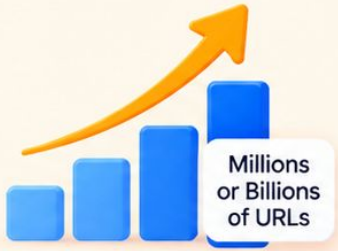
High Availability

System must be available 24/7 with >99.9% uptime.



Performance & Low Latency

URL redirection should occur in milliseconds. Shortening URLs should be near-instantaneous.



Scalability

System must handle millions or billions of URLs, supporting high read volume (redirects) and moderate write volume (URL shortening).



Reliability

Ensure data persistence and no data loss even during failures using durable storage and backups.

Unique URL Generation Strategies

1 RANDOM STRING

Fixed-length random characters



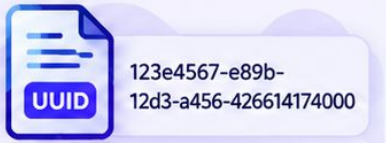
Pros
Unpredictable • no obvious pattern

Trade-off
Collisions possible • handle conflicts

Best for
Unpredictability

2 UUID

128-bit globally unique identifier



Pros
Unique without central coordination

Trade-off
Long and not user-friendly

Best for
Not ideal for short URLs

3 HASH + SALT

Hash the original URL (e.g., SHA-256 + salt)



Pros
Secure • difficult to reverse

Trade-off
May be long • collisions possible • mapping storage needed

Best for
Security-focused cases

RECOMMENDED FOR TinyURL

4 BASE62 ENCODING

Encode an incrementing ID using 0–9, a–z, A–Z



Pros
Short • compact • deterministic • easy to implement

Trade-off
Requires counter/ID management to avoid collisions

Best for
Fast • scalable • short links

Estimating Scale & Identifying Bottlenecks



Estimated User Traffic

- Daily Active Users (DAU): **~10 million**
- Monthly Active Users (MAU): **~300 million**
- New short URLs/day: **~100,000** (1% of DAU)
- Redirect requests/day: **~50 million** (5 per user)



Memory Requirement (Hot URL Cache)

- Cache top **1M** most accessed URLs
- Each mapping ≈ **500 bytes**
- Total memory ≈ **500 MB**



Network Bandwidth (Redirects)

- 50M redirects/day × 700 bytes — **≈ 35 GB/day**
- Average throughput: **~0.4 MB/sec**
- Peak throughput: **~5 MB/sec**



Storage Requirement (URL Mapping DB)

- 100K new URLs/day × 500 bytes — **≈ 50 MB/day**
- Yearly data ≈ **18 GB** raw + overhead
- Plan for **~50 GB/year** with indexes, logs, and backups

Bottleneck Identification

High read volume

Prioritize caching and fast database reads.

Moderate write throughput

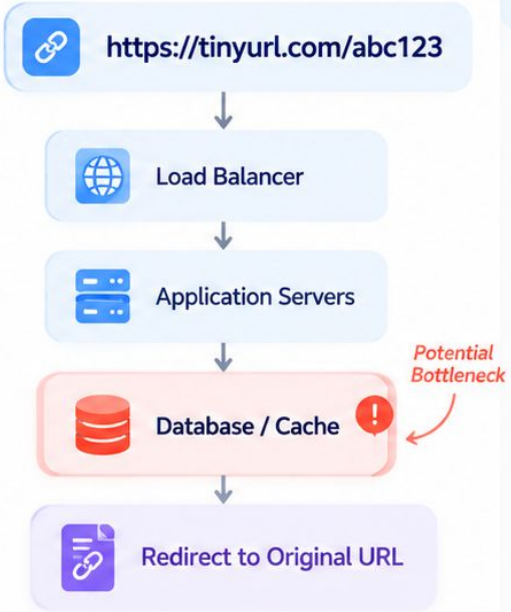
Maintain consistency across writes.

Redirect latency matters

Use low-latency infrastructure.

Traffic bursts happen

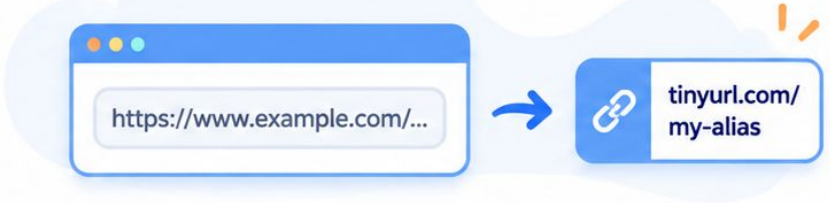
Prepare autoscaling and CDN support.



API Design — Create Short URL

Accepts a long URL and returns a shortened URL.

POST `/api/shorten`



1 Request (JSON)

Send the long URL in the request body.

```json
{
  "long_url": "https://www.example.com/article?id=123"
}
```

TinyURL API

`POST /api/shorten`

2 Response (JSON)

Returns the shortened URL.

```json
{
  "short_url": "https://tinyurl.com/my-alias"
}
```

API Design — Redirect



1. Client Request

User visits the short URL



2. TinyURL Lookup

Look up the original long URL using the short key (abc123).

302 Redirect



3. Redirect to Original URL

Return a 302 HTTP Redirect to the long URL.

Endpoint

GET /:short_key

Example: **GET /abc123**

Behavior

1. Looks up the original long URL using the short key.
2. Returns a 302 HTTP Redirect to the long URL.

API Design — Delete

An optional operation to delete a TinyURL short-link mapping.



Authenticated User

With Bearer token

HTTP Request

DELETE

`/api/url/:short_key`

Verify Ownership

Ensure the authenticated user owns the URL

Mapping Deleted

Short URL mapping is removed from storage

API Endpoint

DELETE `/api/url/:short_key`

Behavior

- Deletes the mapping only when the authenticated user owns the URL.
- Requires a Bearer token for authorization.

User Authentication APIs

1 User Registration

Create a new user account

POST /api/auth/register

2 User Login

Authenticate user and get JWT token

POST /api/auth/login

3 Secure Endpoints

Access protected APIs using Bearer token

```
Authorization: Bearer <access_token>
```

Registration API — POST /api/auth/register

Request:

```
{
  "name": "John Doe",
  "email": "user@example.com",
  "password": "*********"
}
```

Response:

```
{
  "message": "Registration successful"
}
```

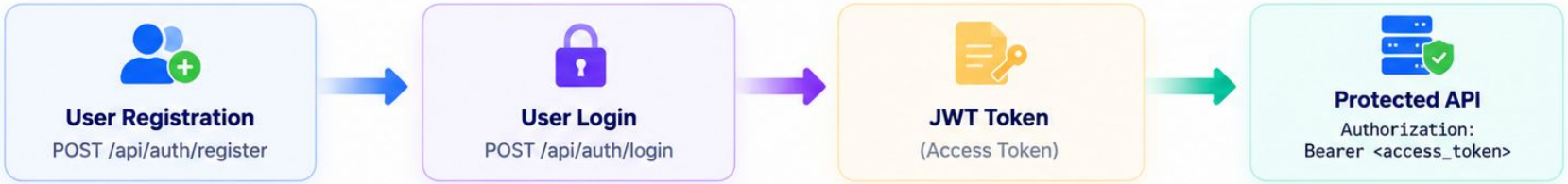
Login API — POST /api/auth/login

Request:

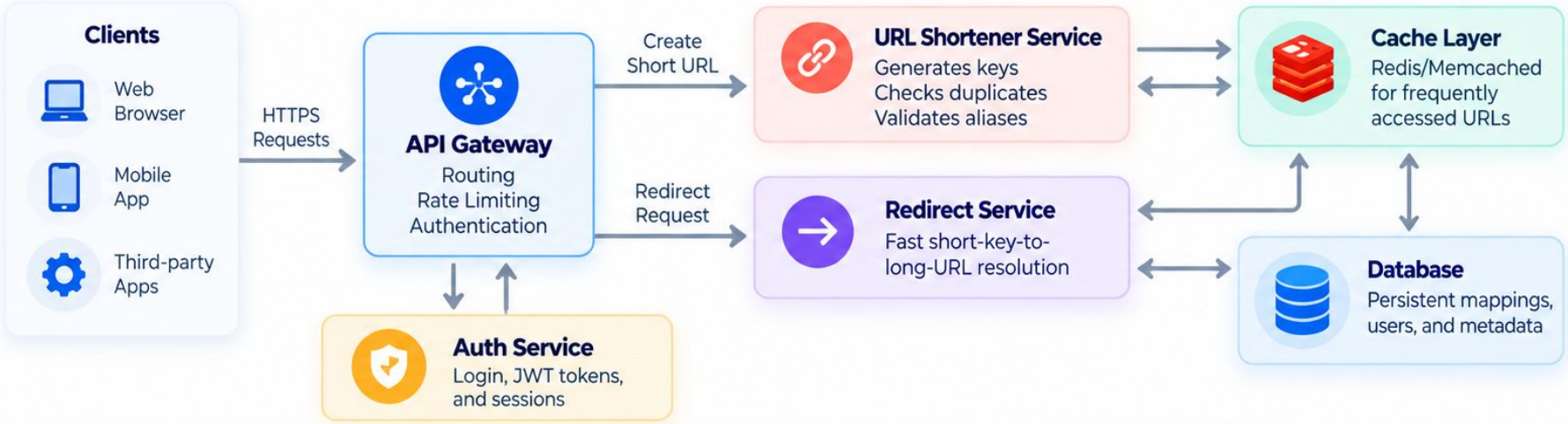
```
{
  "email": "user@example.com",
  "password": "********"
}
```

Response:

```
{
  "access_token": "<JWT>",
  "token_type": "Bearer",
  "expires_in": 3600
}
```



High-Level System Design — Overview



API Gateway
Entry point for clients; routing, rate limiting, authentication.

URL Shortener Service
Generates keys, checks duplicates, validates aliases.

Redirect Service
Fast short-key-to-long-URL resolution.

Database
Persistent mappings, users, and metadata.

Cache Layer
Redis/Memcached for frequently accessed URLs.

Auth Service
Login, JWT tokens, and sessions.

Collision Handling in Distributed URL Generation

Hello ZooKeeper

1 WHY COLLISIONS HAPPEN



- Multiple service instances generating IDs independently → duplicate risk
- Without global coordination, same ID can produce incorrect Base62 URL mapping.

2 WHAT IS ZOOKEEPER?



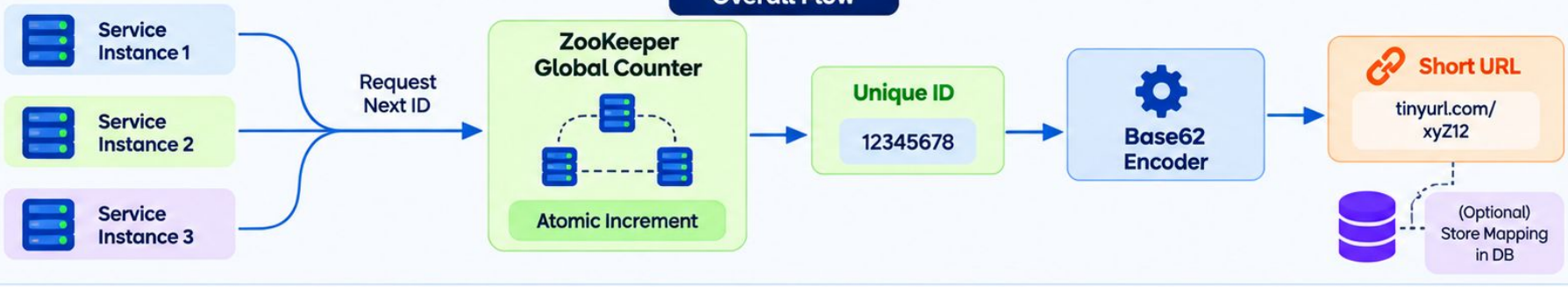
- Apache distributed coordination service
- Synchronizes nodes in a distributed system.

3 ZOOKEEPER AS THE SOLUTION



- Atomic ID generation using ZooKeeper-managed global counter
- Each instance gets a unique ID
- Znodes store/manage counters
- Distributed locks serialize ID generation

Overall Flow



4 FLOW (STEP BY STEP)



Strategic Tech & Infrastructure Decisions for TinyURL

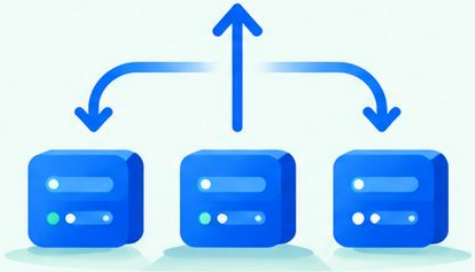
DATABASE STRATEGY

SQL Database
E.g., PostgreSQL with auto-increment IDs.

NoSQL Options
E.g., Redis for caching, DynamoDB for scalability.

Cache
Redis or Memcached for high-speed lookup.

SCALABILITY & PERFORMANCE



Horizontal Scaling

Horizontal scaling for URL generation services.

HIGH AVAILABILITY

Load Balancer
Distribute traffic across service instances.

Database Replication
Replication in DB to avoid single points of failure.

Failover-Ready Infrastructure
Using cloud-managed databases or services.

The Final Design - TinyURL

